C++55期STL部分

Day17

一、六大组件

1、容器：用来存放数据的。将其成为数据结构。

- 序列式容器 vector、list
- 关联式容器 set、map
- 无序关联式容器 unordered_set、unordered_map

2、迭代器：看成是一种广义的指针。可以用来访问容器中的元素。

3、适配器：起到适配的作用。

- 容器适配器 stack、queue、priority_queue
- 迭代器适配器
- 函数适配器 bind

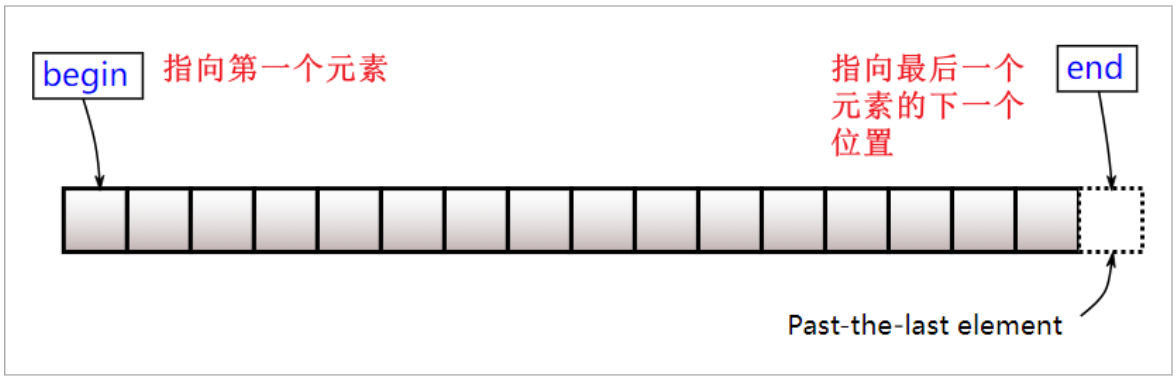
4、算法：普通函数。

5、函数对象（仿函数）：做定制化操作。

6、空间配置器：对空间进行申请与释放。（使用 + 原理 + 源码）

数据结构 + 算法 = 程序

二、序列式容器



1、初始化(掌握)

```
/* vector<int> number;//1、创建无参对象 */
/* vector<int> number(10);//2、count个value值 */
//3、迭代器范围
/* int arr[10] = {1, 3, 5, 7, 9, 8, 6, 4, 2, 10}; */
/* vector<int> number(arr, arr + 10);//左闭右开的区间[arr,arr + 10) */
//4、大括号
vector<int> number = {1, 3, 4, 5, 6, 7, 8, 9, 2, 10};
//5、拷贝或者移动
```

总结：初始化的方法：创建无参对象、count个value、迭代器范围、大括号、拷贝或者移动，三种容器的初始化方式是一样的

2、遍历(掌握)

```cpp
    //遍历
    //下标
    for(size_t idx = 0; idx != number.size(); ++idx)
    {
        cout << number[idx] << "  ";
    }
    cout << endl:

    //未初始化的迭代器
    vector<int>::iterator it;
    for(it = number.begin(); it != number.end(); ++it)
    {
        cout << *it << "  ";
    }
    cout << endl;

    //初始化的迭代器
    vector<int>::iterator it2 = number.begin();
    for(; it2 != number.end(); ++it2)
    {
        cout << *it2 << "  ";
    }
    cout << endl;

    //for与auto结合
    for(auto &elem : number)
    {
        cout << elem << "  ";
    }
    cout << endl;
}
```

总结：遍历的方法可以使用下标、迭代器、以及for循环与auto结合。对于vector与deque而言，几种遍历的方式是完全一样的，但是list不支持下标

```cpp
    //下标
    for(size_t idx = 0; idx != number.size(); ++idx)
    {
        cout << number[idx] << "  ";   list是不支持下标的
    }
    cout << endl;
```

```
wangdao@wangdao:CppBaseDay17$g++ 3_list.cc
3_list.cc: In function 'void test()':
3_list.cc:23:23: error: no match for 'operator[]' (operand types are 's
td::__cxx11::list<int>' and 'size_t' {aka 'long unsigned int'})
   23 |         cout << number[idx] << "  ";
      |                       ^
```

3、在尾部进行插入与删除

```cpp
    cout << endl << "在vector的尾部进行插入与删除元素" << endl;
    number.push_back(100);
    number.push_back(200);
    display(number);
    number.pop_back();
    display(number);
```

总结：三种容器在尾部进行插入与删除是完全一样的。

4、在头部进行插入与删除

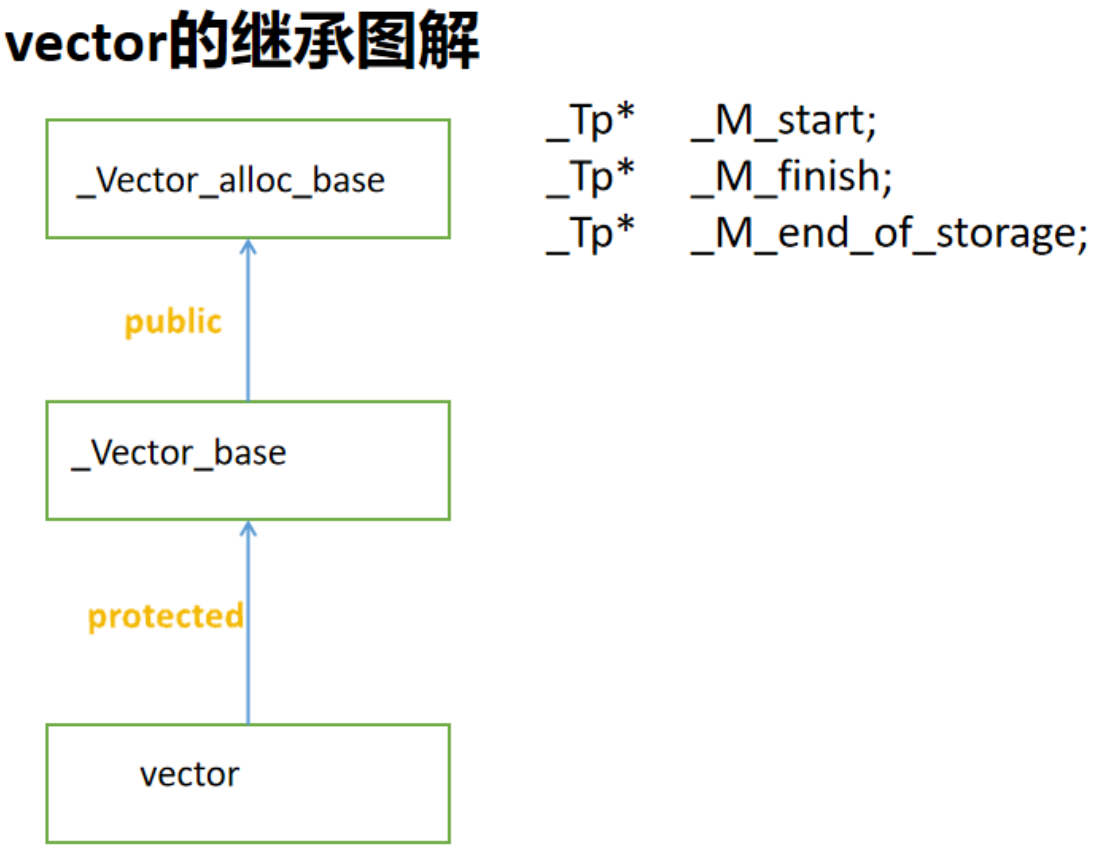
```cpp
    cout << endl << "在list的头部进行插入与删除元素" << endl;
    number.push_front(111);
    number.push_front(222);
    display(number);
    number.pop_front();
    display(number);
```

总结：对于deque与list支持在头部进行插入与删除，但是vector不支持在头部进行插入与删除

```
    //为什么对于vector不允许在头部进行插入与删除呢？
    //因为vector是一端开口的，如果在头部进行插入一个元素或者
    //删除一个元素，会导致后面的元素都需要进行挪动，这样时间
    //复杂度比较高O(N),效率比较低
```

5、vector的源码（了解）

5.1、继承图



5.2、源码

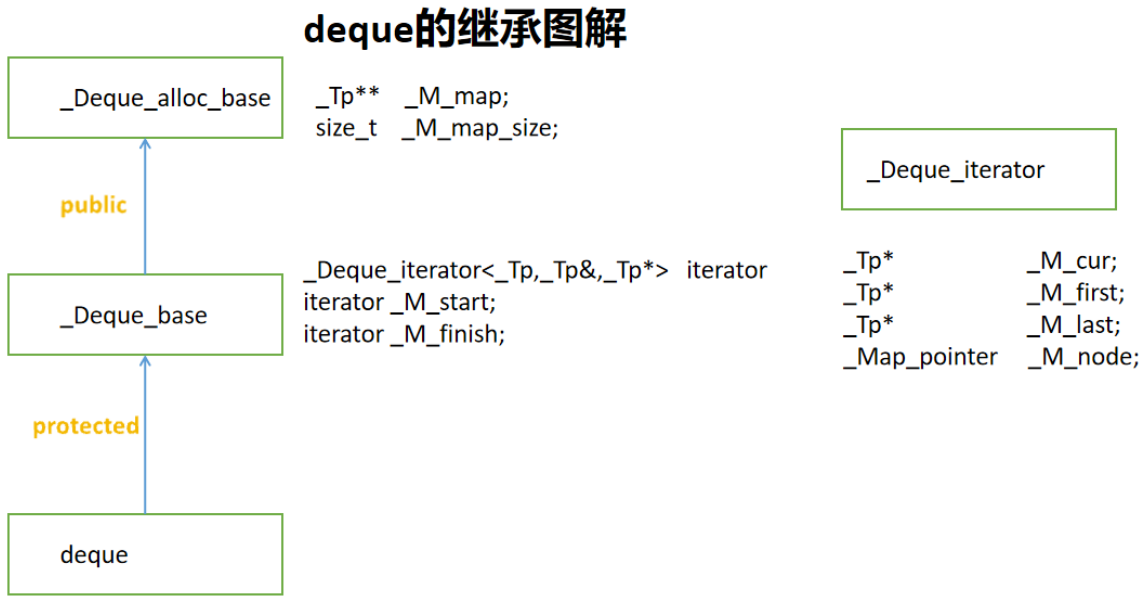
```
template <class _Tp, class _Alloc = __STL_DEFAULT_ALLOCATOR(_Tp) >
class vector
{
public:
  typedef _Tp value_type;
  typedef value_type* pointer;
  typedef const value_type* const_pointer;
  typedef value_type* iterator;
  typedef const value_type* const_iterator;
  typedef value_type& reference;
  typedef const value_type& const_reference;
  typedef size_t size_type;
  typedef ptrdiff_t difference_type;

};
```

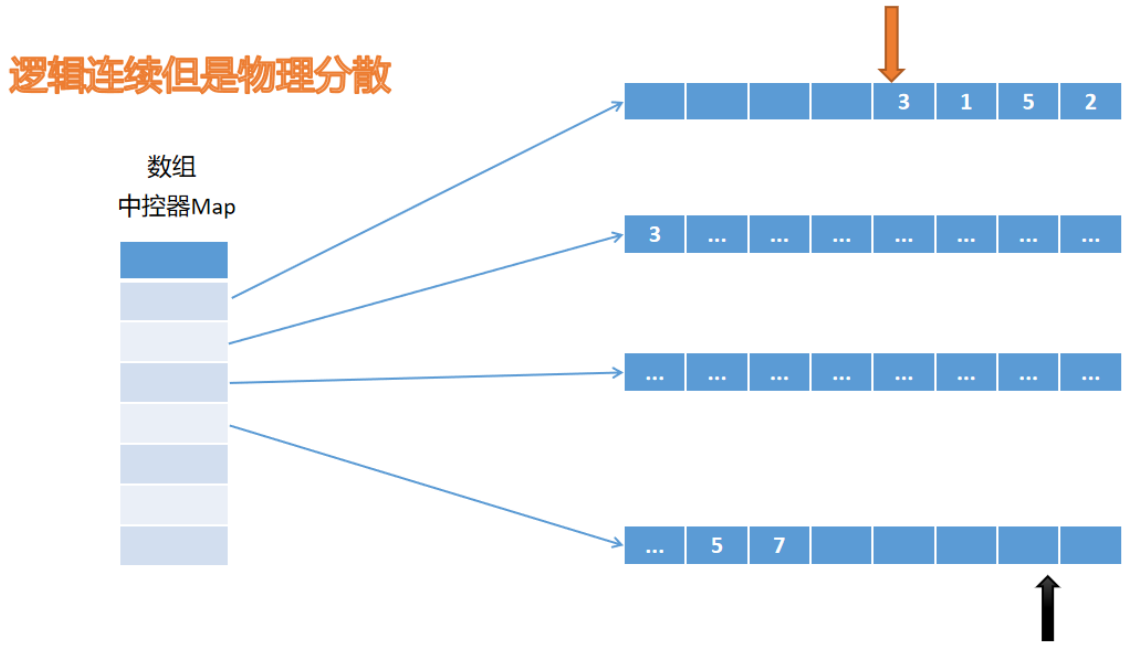
注意：vector中的at函数相较于下标访问运算符而言，更加安全一些，因为具备范围检查。

```
//获取vector中第一个元素的地址
&number;//error,得不到第一个元素的地址
&number[0];//ok
&*number.begin();//ok
int *pdata = number.data();//ok
```

6、deque的源码（了解）

6.1、继承图





7、insert操作(掌握)

std::list<T,Allocator>::insert

```
iterator insert( iterator pos, const T& value );                                  (1)  (until C++11)
iterator insert( const_iterator pos, const T& value );                            (1)  (since C++11)
iterator insert( const_iterator pos, T&& value );                                 (2)  (since C++11)
void insert( iterator pos, size_type count, const T& value );                     (3)  (until C++11)
iterator insert( const_iterator pos, size_type count, const T& value );           (3)  (since C++11)
template< class InputIt >
void insert( iterator pos, InputIt first, InputIt last);                          (4)  (until C++11)
template< class InputIt >
iterator insert( const_iterator pos, InputIt first, InputIt last );               (4)  (since C++11)
iterator insert( const_iterator pos, std::initializer_list<T> ilist );            (5)  (since C++11)
```

1、找一个位置插入一个元素

2、找一个位置插入count个元素

3、找一个位置插入迭代器范围的元素

4、找一个位置插入大括号范围的元素

Inserts elements at the specified location in the container.

```
cout << endl << "在list的任意位置进行插入" << endl;
auto it = number.begin();
++it;
++it;
cout << "*it = " << *it << endl;
number.insert(it, 123);
display(number);
cout << "*it = " << *it << endl;

cout << endl;
number.insert(it, 3, 666);
display(number);
cout << "*it = " << *it << endl;

cout << endl;
vector<int> vec = {11, 33, 22, 44};
number.insert(it, vec.begin(), vec.end());
display(number);
cout << "*it = " << *it << endl;

cout << endl;
number.insert(it, {333, 777, 888});
display(number);
cout << "*it = " << *it << endl;
```

1、找一个位置插入一个元素

2、找一个位置插入count个元素

3、找一个位置插入一个迭代器范围的元素

4、找一个位置插入一个大括号范围的元素

总结：三种序列式容器都具备上述四种插入操作，但是对于vector，有可能会导致迭代器失效，所以需要重点研究一下。

8、vector迭代器失效（重要）

```
//对于vector而言，使用insert操作的时候，有可能会导致底层发生
//扩容操作，导致迭代器已经失效，再用失效的迭代器就有可能发生
//意想不到的的问题
//解决方案：每次使用迭代器的时候，重置迭代器的位置
cout << endl;
it = number.begin();
++it;
++it;
cout << "*it = " << *it << endl;
number.insert(it, 10, 666);
display(number);
cout << "number.size() = " << number.size() << endl;
cout << "number.capacity() = " << number.capacity() << endl;
cout << "*it = " << *it << endl;
```

每次重置迭代器的位置

Day18

一、问题回顾

1、序列式容器的初始化方式有哪五种？遍历方式有哪四种？

2、为何vector没有提供在头部进行插入与删除？

3、deque中的元素是连续的吗？

4、序列式容器在任意位置插入有几种形式？

5、vector迭代器失效问题的原因以及解决方案是什么？

二、序列式容器

1、vector的erase删除（重要）

```
void test2()
{
    vector<int> number = {1, 2, 4, 4, 4, 4, 5, 9};
    for(auto it = number.begin(); it != number.end(); )
    {
        if(4 == *it)
        {
            number.erase(it);
        }
        else
        {
            ++it;
        }
    }

    for(auto it = number.begin(); it != number.end(); ++it)
    {
        cout << *it << "  ";
    }
    cout << endl;
}
```

删除满足条件的元素的时候，后面的元素会前移，如果此时再移动迭代器的位置，会导致漏掉某些元素，特别是删除连续重复元素的时候，删除不干净

只有不删除的时候，才移动迭代器的位置

2、清空元素

```
    cout << endl << "vector清空元素" << endl;
    number.clear();//清空元素
    number.shrink_to_fit();//回收多余的空间
    cout << "number.size() = " << number.size() << endl;
    cout << "number.capacity() = " << number.capacity() << endl;
}
```

```
    cout << endl << "deque清空元素" << endl;
    number.clear();//清空元素
    number.shrink_to_fit();//回收多余的空间
    cout << "number.size() = " << number.size() << endl;
}
```

```
    cout << endl << "list清空元素" << endl;
    number.clear();//清空元素
    cout << "number.size() = " << number.size() << endl;
}
```

总结：三种序列式容器都具备clear函数与size函数，但是对于deque与list而言，没有capacity函数，而对于list而言也没有shrink_to_fit函数。

3、其他操作

swap：可以交换两个容器中的内容。三种序列式容器都具备该函数。

resize：改变元素的个数。三种序列式容器都具备该函数。

front与back：获取容器的第一个元素与最后一个元素。三种序列式容器都具备该函数。

4、emplace_back函数

```
void test00()
{
    vector<Point> vec;
    /* vec.push_back(Point(1, 2)); */
    vec.emplace_back(1, 2);
}
```

调用push_back在此处会先构建一个Point对象，然后将对象拷贝一份到vector中；但是执行emplace_back会直接构建一个Point对象，并将其放置在vector中，所以emplace_back效率要高一些

三、list的特殊操作

1、sort操作

```
    cout << endl << "list的sort操作" << endl;
    /* number.sort(); */
    /* std::less<int> comp; */
    /* number.sort(comp); */
    /* number.sort(std::less<int>()); */
    /* number.sort(std::greater<int>()); */
    number.sort(CompareList());
    display(number);
```

```
struct CompareList
{
    bool operator()(const int &lhs, const int &rhs) const
    {
        cout << "struct CompareList" << endl;
        return lhs < rhs;
    }
};
```

重载函数调用运算符

以后要进行比较大小，可以使用std::less与std::greater。

2、reverse操作

```
    cout << endl << "list的reverse操作" << endl;
    number.reverse();
    display(number);
```

3、unique操作

```
    cout << endl << "list的unique操作" << endl;
    number.unique();
    display(number);
```

删除连续重复元素，如果想将所有重复元素删除，可以先排序，后执行unique

4、merge操作

```
cout << endl << "list的merge操作" << endl;
list<int> number2 = {11, 33, 77, 44, 22, 8};
number2.sort();
display(number2);
number.sort();
display(number);
number.merge(number2);
display(number);
display(number2);
```

合并两个链表的时候，要保证两个链表在合并之前都是按照从小到大的顺序排好序之后，这样在执行完merge之后，合并的链表才是有序的

5、splice操作

```
void splice( const_iterator pos, list& other );
void splice( const_iterator pos, list&& other );
void splice( const_iterator pos, list& other, const_iterator it );
void splice( const_iterator pos, list&& other, const_iterator it );
void splice( const_iterator pos, list& other,
             const_iterator first, const_iterator last);
void splice( const_iterator pos, list&& other,
             const_iterator first, const_iterator last );
```

(1) (since C++11)
(2) (since C++11)
(3) (since C++11)

1、将other中的所有元素进行移动

2、将other中的一个元素进行移动

3、将other中的一个范围的元素进行移动

```
auto it = number.begin();
++it;
++it;
cout << "*it = " << *it << endl;
number.splice(it, number2);
display(number);
display(number2);
```

1、将number2中的所有元素移动到number中移动之后，number2中就没有元素了

```
auto it = number.begin();
++it;
++it;
cout << "*it = " << *it << endl;
auto it2 = number2.end();
--it2;
cout << "*it2 = " << *it2 << endl;
number.splice(it, number2, it2);
display(number);
display(number2);
```

2、将number2中it2指向的元素移动到number的it的前面

```
it = number.begin();
++it;
++it;
cout << "*it = " << *it << endl;
it2 = number2.begin();
++it2;
++it2;
cout << "*it2 = " << *it2 << endl;
auto it3 = number2.end();
--it3;
cout << "*it3 = " << *it3 << endl;
number.splice(it, number2, it2, it3);
display(number);
display(number2);
```

3、将number2中it2到it3范围的元素移动到number的it的前面
注意：[it2, it3),左闭右开的区间

```
122     cout << "*it3 = " << *it3 << endl;
123     number.splice(it, number, it2, it3);
124     display(number);
125
```

splice可以在同一个链表中进行操作，但是不建议it在it2与it3之间

四、set的使用

1、基本特征

```
24      //set的基本特征
25      //1、存放的是key类型，key值是唯一的
26      //2、默认情况下，会按照key值升序排列
27      //3、set的底层实现是红黑树
```

2、查找操作

```
26      cout << endl << "set的查找操作" << endl;
27      size_t cnt = number.count(5);
28      cout << "cnt = " << cnt << endl;
29
30      cout << endl;
31      set<int>::iterator it = number.find(4);
32      /* if(it == nullptr)//error */
33      if(it == number.end())
34      {
35          cout << "该元素不在set中" << endl;
36      }
37      else
38      {
39          cout << "该元素存在set中 " << *it << endl;
40      }
```

3、insert操作

`std::pair<iterator,bool> insert( const value_type& value );`	(1)	
`std::pair<iterator,bool> insert( value_type&& value );`	(2)	(since C++11)
`iterator insert( iterator hint, const value_type& value );`	(3)	(until C++11)
`iterator insert( const_iterator hint, const value_type& value );`		(since C++11)
`iterator insert( const_iterator hint, value_type&& value );`	(4)	(since C++11)
`template< class InputIt >` `void insert( InputIt first, InputIt last );`	(5)	
`void insert( std::initializer_list<value_type> ilist );`	(6)	(since C++11)

```cpp
    cout << endl << "set的insert操作" << endl;
    pair<set<int>::iterator, bool> ret = number.insert(5);
    if(ret.second)
    {
        cout << "插入成功 " << *ret.first << endl;
    }
    else
    {
        cout << "插入失败, 该元素存在set中" << endl;
    }
    display(number);

    cout << endl;
    vector<int> vec = {11, 8, 9, 4, 2, 10};
    number.insert(vec.begin(), vec.end());
    display(number);

    cout << endl;
    number.insert({5, 20, 40, 12});
    display(number);
```

4、不支持下标

```cpp
    cout << endl << "set的下标操作" << endl;
    cout << "number[1] = " << number[1] << endl;
}
                                set不支持下标操作
int main(int argc, char *argv[])
{
```

```
wangdao@wangdao:CppBaseDay18$g++ 6_set.cc
6_set.cc: In function 'void test()':
6_set.cc:75:37: error: no match for 'operator[]' (operand types are
std::set<int>' and 'int')
   75 |     cout << "number[1] = " << number[1] << endl;
      |                                     ^
wangdao@wangdao:CppBaseDay18$
```

5、不支持使用迭代器修改

```cpp
    cout <<endl << "set的修改" << endl;
    it = number.begin();
    cout << "*it = " << *it << endl;
    /* *it = 100;//error */
}

int main(int argc, char *argv[])
```

```
*it = 1
wangdao@wangdao:CppBaseDay18$g++ 6_set.cc
6_set.cc: In function 'void test()':
6_set.cc:80:9: error: assignment of read-only location 'it.std::_Rb_t
ree_const_iterator<int>::operator*()'
   80 |     *it = 100;
      |     ~~~~^~~~
wangdao@wangdao:CppBaseDay18$
```

6、针对于自定义类型（重要）

Defined in header <set>

```cpp
template<
    class Key,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<Key>
> class set;
```

针对于自定义类型而言，需要自己实现Compare进行比较Key

```cpp
//命名空间的扩展
namespace  std
{

//模板的特化
template <>
struct less<Point>
{
    bool operator()( const Point& lhs, const Point& rhs ) const
    {
        cout << "struct less<Point>" << endl;
        if(lhs.getDistance() < rhs.getDistance())
        {
```

1、模板的特化

```
//函数对象的形式
struct ComparePoint
{
    bool operator()( const Point& lhs, const Point& rhs ) const
    {
        cout << "struct ComparePoint" << endl;
        if(lhs.getDistance() < rhs.getDistance())
        {
```

2、函数对象的形式

```

//运算符的重载
bool operator<(const Point &lhs, const Point &rhs)
{
    cout << "bool operator<()" << endl;
    if(lhs.getDistance() < rhs.getDistance())
    {
```

3、运算符的重载

总结：针对于自定义类型而言，可以使用模板的特化、函数对象的形式、运算符重载。

五、multiset的使用

1、基本特征

```
    //multiset的基本特征
    //1、存放的是key类型，key值是不唯一的，可以重复
    //2、默认情况下，会按照key值升序排列
    //3、multiset的底层实现是红黑树
    multiset<int> number = {1, 2, 4, 7, 9, 6, 4, 3, 3};
    display(number);

```

2、其他操作

multiset的查找操作count与find与set的使用完全一样。插入操作insert与set基本一样。也不支持下标访问，也不能使用迭代器进行修改元素。

3、bound函数

```
    cout << endl << "multiset的bound操作" << endl;
    auto it2 = number.lower_bound(4);
    cout << "*it2 = " << *it2 << endl;
    auto it3 = number.upper_bound(4);
    cout << "*it3 = " << *it3 << endl;

    pair<multiset<int>::iterator, multiset<int>::iterator>
        ret = number.equal_range(4);
    while(ret.first != ret.second)
    {
        cout << *ret.first << "  ";
        ++ret.first;
    }
    cout << endl;
```

返回第一个不小于给定key的迭代器

返回第一个大于给定key的迭代器

返回的是一个范围，第一个迭代器指向的是大于等于给定key的迭代器第二个迭代器是大于给定key的迭代器

4、针对于自定义类型

Defined in header <set>

```
template<
    class Key,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<Key>
> class multiset;
```

针对于自定义类型而言，multiset也有三种形式：模板的特化、函数对象的形式、运算符重载的形式。

Day19

一、问题回顾

1、vector删除连续重复元素的时候，要注意什么？

2、序列式容器的基本操作有哪些：在尾部插入与删除、在头部插入与删除、在任意位置插入、删除、清空元素、交换操作、重置元素个数、获取元素个数、获取首元素与尾部元素等？

3、list有哪些特殊操作？需要注意什么？

4、关联式容器set与multiset的特点是什么？有哪些基本操作？

5、set与multiset针对于自定义类型有三种写法？

二、map的使用

1、基本特征

```
    //map的特征
    //1、存放的是key-value类型，key值是唯一的，不能重复,value
    //值可以重复，也可以不重复
    //2、默认会按照key值进行升序排列
    //3、底层使用的是红黑树
    map<int, string> number = {
        {1, "wuhan"},
        {3, "hubei"},
        pair<int, string>(5, "beijing"),
        pair<int, string>(4, "beijing"),
        make_pair(6, "shanghai"),
        make_pair(10, "nanjing"),
        make_pair(1, "nanjing"),
    };
    display(number);
```

2、查找操作

```
    cout << endl << "map的查找操作" << endl;
    size_t cnt = number.count(5);
    cout << "cnt = " << cnt << endl;

    auto it = number.find(6);
    if(it != number.end())
    {
        cout << "该元素存在map中 "
             << it->first << "  "
             << it->second << endl;
    }
    else
    {
        cout << "该元素不在map中" << endl;
    }
```

3、插入操作

```
    cout << endl << "map的insert操作" << endl;
    pair<map<int, string>::iterator, bool> ret =
        number.insert({8, "dongjing"});
    /* number.insert(pair<int, string>(8, "dongjing")); */
    /* number.insert(make_pair(8, "dongjing")); */
    if(ret.second)
    {
        cout << "插入成功 "
             << ret.first->first << "  "
             << ret.first->second << endl;
    }
    else
    {
        cout << "插入失败,该元素存在map中" << endl;
    }
    display(number);
```

4、下标操作（重要）

```
    cout << endl << "map的下标操作" << endl;
    cout << "number[1] = " << number[1] << endl;//查找
    cout << "number[7] = " << number[7] << endl;//插入
    display(number);

    cout << endl;
    //T &operator[](const Key &)
    number[7] = "wangdao";//修改
    number[1] = "wangdao";
    display(number);

    const map<int, int> tmp = {{1, 3}};
    /* cout << tmp[1] << endl;//error */
```

5、自定义类型

```
template<
    class Key,
    class T,
    class Compare = std::less<Key>
    class Allocator = std::allocator<std::pair<const Key, T> >
> class map;
```

针对的是key类型进行比较

如果key是类类型，但是如果自身已经可以比较大小，那么针对于自定义类型而言，就无需要改动了，也就是不需要模板的特化、函数对象的形式、运算符重载这些操作。

三、multimap的使用

1、基本特征

```
    //multimap的特征
    //1、存放的是key-value类型，key值是不唯一的，可以重复，
    //value值可以重复，也可以不重复
    //2、默认会按照key值进行升序排列
    //3、底层使用的是红黑树
    multimap<int, string> number = {
        {1, "wuhan"},
        {3, "hubei"},
        pair<int, string>(5, "beijing"),
        pair<int, string>(4, "beijing"),
        make_pair(6, "shanghai"),
        make_pair(10, "nanjing"),
        make_pair(1, "nanjing"),
    };
    display(number);
```

2、其他操作

multimap的查找操作count与find与map是一样的。multimap进行插入的时候，肯定是可以插入成功的，所以返回类型与map的insert稍微有点区别。

3、不支持下标

```
    cout << endl << "multimap的下标操作" << endl;
    cout << "number[1] = " << number[1] << endl;//error
    cout << "number[7] = " << number[7] << endl;//error
    display(number);
```

因为multimap中key值可以重复，为了避免通过同一个key返回不同的value而导致二义性，所以不提供下标

总结：

1、所有关联式容器中的key值都是有序的。

2、所有关联式容器的底层使用的都是红黑树。

3、set与map的key是唯一的，不能重复。而multiset与multimap的key是可以重复的。

4、关联式容器中只有map具备下标。

四、哈希

1、哈希函数

通过key值找到key在哈希表中的位置。

```
index = H(key)
```

2、哈希函数的构建方式

1、直接定址法：　H(key) = a * key + b

2、平方取中法：　key^2 = 1234^2 = 1522756 ------>227

3、数字分析法：H(key) = key % 10000；

4、除留取余法：H(key) = key mod p　(p <= m, m为表长)

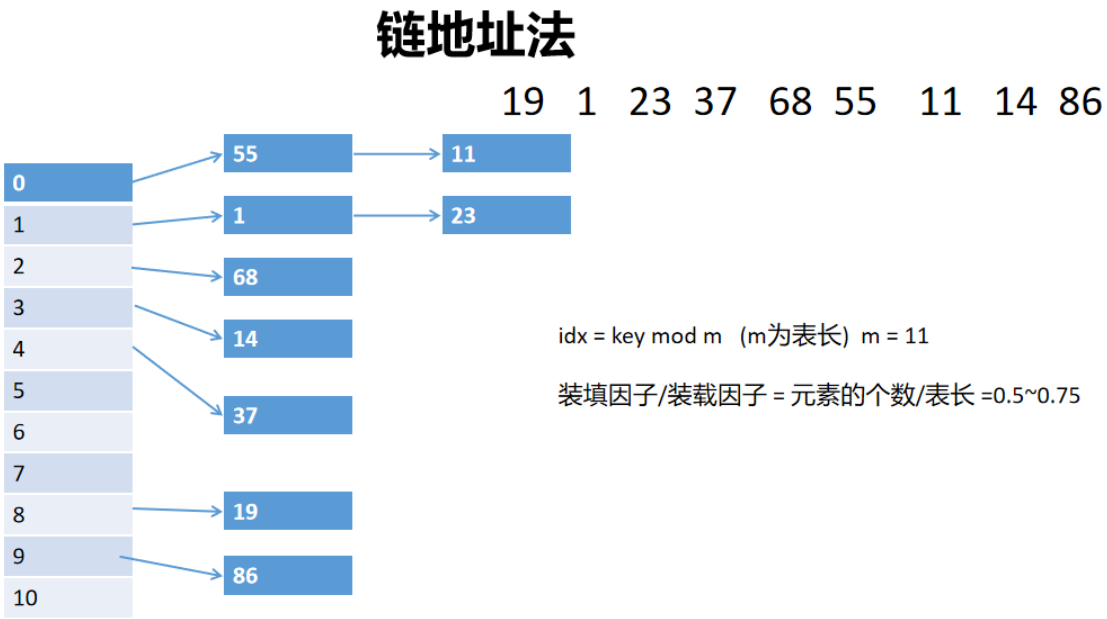
3、哈希冲突

对于不同的key值，经过哈希函数运算之后，会存在同一个位置，这就是哈希冲突。

```
H(key1) = H(key2),    如果key1 != key2
```

4、解决哈希冲突的方法

开放定址法、链地址法 (推荐使用这种，这也是STL中使用的方法)、再散列法、建立公共溢出区



5、装载因子（装填因子）

元素的个数 / 表的长度 ＝ 装填因子，装填因子的值越大，发生冲突的概率要高，空间的利用率也是要高；装填因子的值越小，发生冲突的概率要低，空间的利用率也是要低；0.5~0.75。数组可以看成是一个完美的哈希。

五、unordered_set的使用

1、模板参数

Defined in header <unordered_set>

```
template<
    class Key,
    class Hash = std::hash<Key>,
    class KeyEqual = std::equal_to<Key>,
    class Allocator = std::allocator<Key>
> class unordered_set;
```

2、基本特征

```
    //unordered_set的特征
    //1、存放的是key类型，key值是唯一的，不能重复
    //2、key值是没有顺序的
    //3、底层使用的是哈希
    unordered_set<int> number = {1, 3, 9, 8, 6, 4, 3, 2, 2};
    display(number);
```

3、其他操作

unordered_set的count、find、insert、erase操作与set是完全一样的。也不支持下标操作。

4、针对于自定义类型（重要）

对于模板参数的第二个Hash,针对于自定义类型而言，有两种方法：模板的特化和函数对象的形式。

```
namespace  std
{

//模板的特化
template <>
struct hash<Point>
{
    size_t operator()(const Point &rhs) const
    {
        cout << "struct hash<Point>" << endl;
        return (rhs.getX() << 2)^(rhs.getY() << 2);
    }
};

}//end of namespace std
```

1、std::hash针对于自定义类型的模板特化形式

```
//函数对象的形式
struct HashPoint
{
    size_t operator()(const Point &rhs) const
    {
        cout << "struct HashPoint" << endl;
        return (rhs.getX() << 2)^(rhs.getY() << 2);
    }

};
```

2、函数对象的形式

对于模板参数的第三个KeyEqual,针对于自定义类型而言，有三种方法：模板的特化、函数对象的形式、运算符重载。

```
//模板的特化形式
namespace  std
{
template <>
struct equal_to<Point>
{
    bool operator()( const Point& lhs, const Point& rhs ) const
    {
        cout << "struct equal_to<Point>" << endl;
        return ((lhs.getX() == rhs.getX())
                && (lhs.getY() == rhs.getY()));
    }
};

}//end of namespace std
```

1、模板的特化

```
struct EqualToPoint
{
    bool operator()( const Point& lhs, const Point& rhs ) const
    {
        cout << "struct EqualToPoint" << endl;
        return ((lhs.getX() == rhs.getX())
                && (lhs.getY() == rhs.getY()));
    }

};
```

2、函数对象的形式

```
//运算符的重载
bool operator==(const Point &lhs, const Point &rhs)
{
    cout << "bool operator==()" << endl;
    return ((lhs.getX() == rhs.getX())
            && (lhs.getY() == rhs.getY()));
}
```

3、运算符重载

Day20

一、问题回顾

1、关联式容器有哪些？有什么特点？

2、什么是哈希函数？什么是哈希冲突?什么是装载因子？

二、文本查询扩展作业讲解

```
get_file函数的作用：就是打开文件，然后将文件中的每一行的内容放在数据结构shared_ptr<vector<string>> file中存起来；然后将单词与单词所对应的行号放在数据结构map<string, shared_ptr<set<size_t>>> wm里面存起来。
```

Query

```
class Query
{
public:
   Query(const std::string &s)
   : q(new WordQuery(s))
   {
   }
private:
     Query(std::shared_ptr<Query_base> query)
    : q(query)
    {

    }
private:
    shared_ptr<Query_base> q;
};

shared_ptr<Query_base>  q(new WordQuery(s));
```

Query_base

```
class Query_base
{
    friend class Query;
protected:
    typedef TextQuery::line_no line_no; // used in the eval functions
    virtual ~Query_base() { }
private:
    virtual QueryResult eval(const TextQuery&) const = 0;
    virtual std::string rep() const = 0;
};
```

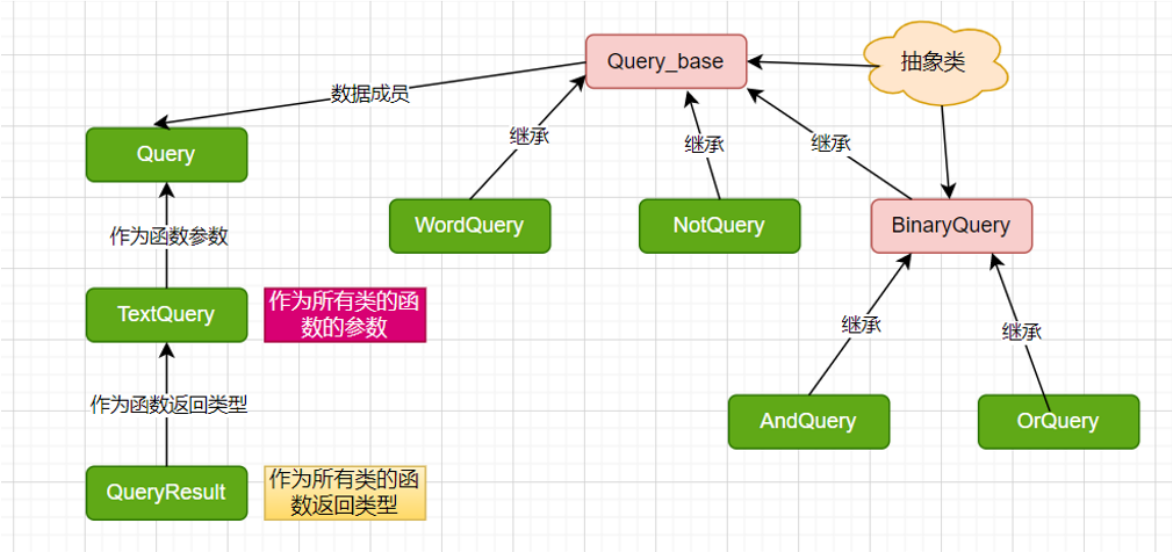
查询单词所做的操作

```
将待查询的单词交给sought，然后通过sought构建一个Query对象，在构建Query对象的时候，会使用sought去构建一个WordQuery对象，并且在Query对象中存在数据成员shared_ptr<Query_base> q指向new WordQuery对象。然后走Query对象的eval函数，该函数会体现多态，调用WordQuery的eval方法（此处要数据动态多态被激活的五个条件），而在WordQuery的eval方法会调用TextQuery中的query函数，而query函数的作用是将待查询的单词sought、单词的行号、每一行的内容会存放在QueryResult里面，最终将QueryResult对象进行打印即可。
```

查询两个单词同时出现的操作

```
待查询的单词是sought1与sought2，
然后使用sought1构建Query对象，该对象中会有一个数据成员shared_ptr<Query_base> q(new WordQuery(sought1));
使用sought2构建Query对象，该对象中会有一个数据成员shared_ptr<Query_base> q(new WordQuery(sought2));
在将两个Query进行与操作的时候会返回一个对象andq，里面也会有数据成员shared_ptr<Query_base> q(new AndQuery(lhs, rhs));

Query andq = Query(sought1)  & Query(sought2)
inline Query operator&(const Query &lhs, const Query &rhs)
{
    return std::shared_ptr<Query_base>(new AndQuery(lhs, rhs));//隐式转换
    //shared_ptr<Query_base>  t(new AndQuery(lhs, rhs));
    //return t;  Query(t)
}

Query(shared_ptr<Query_base>);

最终走AndQuery的eval方法，也就是取交集。
按照这里逻辑，如果是查询两个词，至少出现一个，会走OrQuery的eval方法，然后取并集。
如果是查询某个词没有出现，会走NotQuery的eval方法，然后取差集。
```

文本查询扩展类之间关系



三、unordered_multiset的使用

1、基本特征

```
void test()
{
    //unordered_multiset的特征
    //1、存放的是key类型，key值是不唯一的，可以重复
    //2、key值是没有顺序的
    //3、底层使用的是哈希
    unordered_multiset<int> number = {1, 3, 9, 8, 6, 4, 3, 2, 2
    display(number);
}
```

2、其他操作

unordered_multiset的查找count和find、insert、erase与unordered_set基本一致。

3、针对于自定义类型

Defined in header <unordered_set>

```
template<
    class Key,
    class Hash = std::hash<Key>,
    class KeyEqual = std::equal_to<Key>,
    class Allocator = std::allocator<Key>
> class unordered_multiset;
```

对于自定义类型而言，也需要实现第二个模板参数Hash与第三个模板参数KeyEqual，实现方式与unordered_set完全一样。

四、unordered_map的使用

1、基本特征

```
    //unordered_map的特征
    //1、存放的是key-value类型，key值是唯一的，不能重复，但是
    //value值可以重复，也可以不重复
    //2、key值是没有顺序的
    //3、底层实现是哈希
    unordered_map<int, string> number = {
        {1, "hello"},
        {4, "hello"},
        make_pair(5, "wangdao"),
        make_pair(3, "wuhan"),
        pair<int, string>(4, "wangdao"),
        pair<int, string>(6, "beijing"),
    };
    display(number);
```

2、下标

```
    cout << endl << "unordered_map的下标" << endl;
    cout << "number[1] = " << number[1] << endl;//查找
    cout << "number[8] = " << number[8] << endl;//插入
    display(number);

    cout << endl;
    number[8] = "wangdao";//修改
    display(number);
```

3、针对于自定义类型

Defined in header <unordered_map>

```
template<
    class Key,
    class T,
    class Hash = std::hash<Key>,
    class KeyEqual = std::equal_to<Key>,
    class Allocator = std::allocator< std::pair<const Key, T> >
> class unordered_map;
```

Defined in header <string>

```
template<> struct hash<std::string>;
template<> struct hash<std::wstring>;
template<> struct hash<std::u8string>; // c++20      (since C++11)
template<> struct hash<std::u16string>;
template<> struct hash<std::u32string>;

template<> struct hash<std::pmr::string>;
template<> struct hash<std::pmr::wstring>;
template<> struct hash<std::pmr::u8string>;          (since C++20)
template<> struct hash<std::pmr::u16string>;
template<> struct hash<std::pmr::u32string>;
```

五、unordered_multimap的使用

1、基本特征

```
    //unordered_multimap的特征
    //1、存放的是key-value类型，key值是不唯一的，可以重复，但是
    //value值可以重复，也可以不重复
    //2、key值是没有顺序的
    //3、底层实现是哈希
    unordered_multimap<int, string> number = {
        {1, "hello"},
        {4, "hello"},
        make_pair(5, "wangdao"),
        make_pair(3, "wuhan"),
        pair<int, string>(4, "wangdao"),
        pair<int, string>(6, "beijing"),
    };
    display(number);
```

总结：

1、所有无序关联式容器中的key值都是没有序的。

2、所有无序关联式容器的底层使用的都是哈希。

3、unordered_set与unordered_map的key是唯一的，不能重复。而unordered_multiset与unordered_multimap的key是可以重复的。

4、无序关联式容器中只有unordered_map具备下标。

六、容器的选择（重要）

1、有没有顺序

如果是有序的，首先选择的是关联式容器，不应该选择的是无序关联式容器；其次才是序列式容器（list有成员函数sort，vector与deque在算法库中有sort函数让其中的元素进行排序）

2、有没有下标

序列式容器：vector、deque

关联式容器：map

无序关联式容器：unordered_map

vector与deque的下标具备随机访问的含义，但是map与unordered_map的下标使用的是key类型。

3、查找的时间复杂度

序列式容器：只能通过遍历的方法进行查询，查询的时间复杂度O(N)。

关联式容器：因为底层使用的是红黑树，查询的时间复杂度O(logN)。

无序关联式容器：因为底层使用的是哈希，查询的时间复杂度O(1)

4、迭代器的类型

随机访问迭代器：vector、deque

双向迭代器：list、关联式容器

前向迭代器：无序关联式容器

注意：容器适配器也可以当容器进行看待，但是容器适配器是没有迭代器的。

七、priority_queue的使用

1、模板参数

Defined in header <queue>

```
template<
    class T,
    class Container = std::vector<T>,
    class Compare = std::less<typename Container::value_type>
> class priority_queue;
```

萃取技术

A priority queue is a container adaptor that provides constant time lookup of the largest (by a

2、原理

```
{
    //优先级队列底层会用到堆排序：默认使用的是大根堆
    //当有元素插入进来的时候，会将堆顶与新插入的元素进行比较，
    //如果堆顶比新插入的元素要小，就满足了std::less,就会将
    //新插入的元素作为新的堆顶；如果堆顶比新插入的元素要大
    //就不满足std::less,那么上一次堆顶依旧是新的堆顶
    vector<int> vec = {1, 3, 5, 8, 6, 4, 9, 7, 3};
    priority_queue<int> pque;

    for(size_t idx = 0; idx != vec.size(); ++idx)
    {
        pque.push(vec[idx]);
        cout << "优先级最高的元素是: " << pque.top() << endl;
    }

    cout << endl <<endl;
    while(!pque.empty())
    {
        cout << pque.top() << "  ";
        pque.pop();
    }
    cout << endl;
```

3、针对于自定义类型

针对于自定义类型而言，需要比较Contaner::value_type，如果是一个自定义类型的话，需要改写。有三种方法：模板的特化、函数对象的形式、运算符重载。（与以前set中的模板参数Compare是完全一样的）

Day21

一、问题回顾

1、无序关联式容器有哪些？有什么特点？

2、无序关联式容器中，针对于自定义类型而言，哪些模板参数需要进行改写？如何改写？

3、优先级队列底层使用什么？

4、如何选择合适的容器？

二、迭代器

1、基本概念

迭代器可以指向容器中的元素，然后进行移动迭代器的位置，进而达到访问容器中的所有元素。可以将容器看成是一种广义的指针。也就是具备指针的功能。

2、迭代器的类型

输入迭代器、输出迭代器、前向迭代器、双向迭代器、随机访问迭代器

3、输出流迭代器

```
template <class _Tp,
          class _CharT = char, class _Traits = char_traits<_CharT> >
class ostream_iterator
{
public:
    //ostream_iterator<int> osi(cout, "\n");
     ostream_iterator(ostream_type& __s, const _CharT* __c)
    : _M_stream(&__s)//_M_stream = &cout;
    , _M_string(__c)  //_M_string = "\n"
    {}

     ostream_iterator<_Tp>& operator=(const _Tp& __value)
     {
         *_M_stream << __value;//cout << __value;
         if (_M_string) *_M_stream << _M_string;//cout << _M_string;
         return *this;
     }

    ostream_iterator<_Tp>& operator*() { return *this; }
    ostream_iterator<_Tp>& operator++() { return *this; }
    ostream_iterator<_Tp>& operator++(int) { return *this; }
private:
  ostream_type* _M_stream;
  const _CharT* _M_string;
};

//copy(vec.begin(), vec.end(), osi);
//first = vec.begin();
//last = vec.end();
//d_first = osi;
           last
1, 3, 7, 5
           f
template<class InputIt, class OutputIt>
OutputIt copy(InputIt first, InputIt last, OutputIt d_first)
{
    while (first != last)
    {
        *d_first++ = *first++;
    }
    return d_first;
}

osi = 3;
```

```
    vector<int> vec = {1, 3, 7, 5};
    ostream_iterator<int> osi(cout, " ");
    copy(vec.begin(), vec.end(), osi);
```

4、输入流迭代器

```
template <class _Tp,
          class _CharT = char, class _Traits = char_traits<_CharT>,
          class _Dist = ptrdiff_t>
class istream_iterator {
public:
    istream_iterator()
```

```
    : _M_stream(0)
    , _M_ok(false)
    {}
    //istream_iterator<int> isi(cin);
    istream_iterator(istream_type& __s)
    : _M_stream(&__s) //_M_stream = &cin;
    {
        _M_read();
    }

     void _M_read()
     {
         _M_ok = (_M_stream && *_M_stream) ? true : false;
         if (_M_ok)
         {
             *_M_stream >> _M_value;//  cin >> _M_value;
             _M_ok = *_M_stream ? true : false;
         }
     }
    bool _M_equal(const istream_iterator& __x) const
    {
        return (_M_ok == __x._M_ok) && (!_M_ok || _M_stream == __x._M_stream);
    }

    reference operator*() const
    {
        return _M_value;
    }
    istream_iterator& operator++()
    {
        _M_read();
        return *this;
    }
    istream_iterator operator++(int)
    {
        istream_iterator __tmp = *this;
        _M_read();
        return __tmp;
    }
private:
    istream_type* _M_stream;
    _Tp _M_value;
    bool _M_ok;

};

//first = isi;
//last = istream_iterator<int>();
//d_first = std::back_inserter(vec));
copy(isi, istream_iterator<int>(), std::back_inserter(vec));
OutputIt copy(InputIt first, InputIt last, OutputIt d_first)
{
    while (first != last)
    {
        *d_first++ = *first++;
    }
    return d_first;
```

```
}

 d_first  =   2;


inline bool
operator!=(const istream_iterator& __x, const istream_iterator& __y)
{
    return !__x._M_equal(__y);
}

bool _M_equal(const istream_iterator& __y) const
{
    return (_M_ok == __y._M_ok) && (!_M_ok || _M_stream == __y._M_stream);
}

template <class _Container>
inline back_insert_iterator<_Container> back_inserter(_Container& __x) {
  return back_insert_iterator<_Container>(__x);
}

class back_insert_iterator {
protected:
  _Container* container;
public:
    back_insert_iterator<_Container>&
    operator=(const typename _Container::value_type& __value)
    {
        container->push_back(__value);
        return *this;
    }
  back_insert_iterator<_Container>& operator*() { return *this; }
  back_insert_iterator<_Container>& operator++() { return *this; }
  back_insert_iterator<_Container>& operator++(int) { return *this; }
};
```

```
void test()
{
    vector<int>  vec;
    istream_iterator<int> isi(cin);
    cout << "111" << endl;
    //back_inserter函数会返回类类型back_insert_iterator，该类的
    //底层会调用push_back函数
    copy(isi, istream_iterator<int>(), std::back_inserter(vec));
    copy(vec.begin(), vec.end(),
         ostream_iterator<int>(cout, "  "));
    cout << endl;
}
```

三、插入迭代器（掌握）

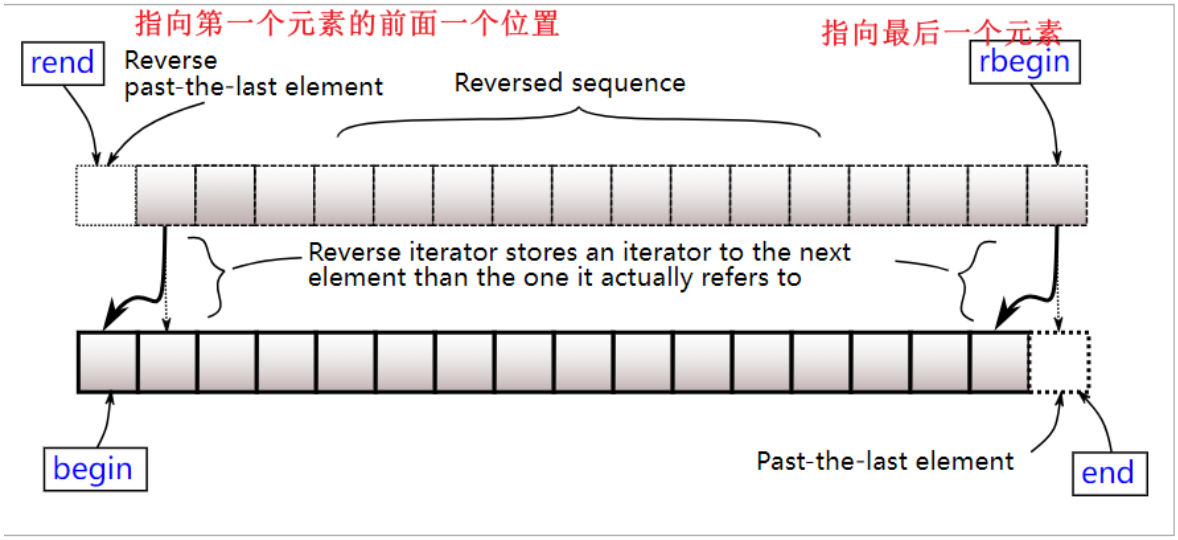
back_inserter是函数模板，该函数模板返回类型是back_insert_iterator，该类是一个类模板，底层会调用push_back。

front_inserter是函数模板，该函数模板返回类型是front_insert_iterator，该类是一个类模板，底层会调用push_front。

inserter是函数模板，该函数模板返回类型是insert_iterator，该类是一个类模板，底层会调用insert。

```
24      vector<int> vec = {1, 3, 7, 9, 6};
25      list<int> lst = {22, 44, 77, 33, 11};
26      //back_inserter返回类型是back_insert_itsrator,底层会调用
27      //push_back
28      /* copy(lst.begin(), lst.end(), back_inserter(vec)); */
29      copy(lst.begin(), lst.end(),
30           back_insert_iterator<vector<int>>(vec));
31
32      copy(vec.begin(), vec.end(),
33           ostream_iterator<int>(cout, "  "));
34      cout << endl;

36      //将vector中的元素插入到list的头部
37      cout << endl;
38      //front_inserter返回类型是front_insert_itsrator,底层会调用
39      //push_front
40      /* copy(vec.begin(), vec.end(), front_inserter(lst)); */
41      copy(vec.begin(), vec.end(),
42           front_insert_iterator<list<int>>(lst));
43
44      copy(lst.begin(), lst.end(),
45           ostream_iterator<int>(cout, "  "));
46      cout << endl;

48      cout << endl;
49      set<int> setNum = {1, 3, 9, 44, 33, 11, 55};
50      auto it = setNum.begin();
51      //inserter返回类型是insert_itsrator,底层会调用insert
52      /* copy(vec.begin(), vec.end(), inserter(setNum, it)); */
53      copy(vec.begin(), vec.end(),
54           insert_iterator<set<int>>(setNum, it));
55      copy(setNum.begin(), setNum.end(),
56           ostream_iterator<int>(cout, "   "));
57      cout << endl;
```

四、反向迭代器



```
void test()
{
    vector<int> vec = {1, 4, 7, 9, 8, 5};
    vector<int>::reverse_iterator rit = vec.rbegin();
    for(; rit != vec.rend(); ++rit)
    {
        cout << *rit << "  ";
    }
    cout << endl;
}
```

五、算法

1、头文件

```
#include <algorithm>
```

2、分类

- 非修改式的算法 **for_each**、count、find
- 修改式的算法 copy、**remove_if**
- 排序算法 sort
- 二分搜索 lower_bound、upper_bound
- 集合操作 set_intersection
- 堆操作 make_heap、push_heap、pop_heap
- 取最值 max、min
- 数值操作 accumulate
- 未初始化的内存操作 uninitialized_copy

3、for_each的用法

一元函数：函数的参数只有一个。二元函数：函数的参数是两个。

一元谓词：函数的参数只有一个，并且函数的返回类型是bool。二元谓词：函数的参数是两个，并且函数的返回类型是bool。

Defined in header <algorithm>

```
template< class InputIt, class UnaryFunction >
UnaryFunction for_each( InputIt first, InputIt last, UnaryFunction f );
template< class InputIt, class UnaryFunction >
constexpr UnaryFunction for_each( InputIt first, InputIt last, UnaryFunction f );
```

```
template <class _InputIter, class _Function>
_Function for_each(_InputIter __first, _InputIter __last, _Function __f) {
  __STL_REQUIRES(_InputIter, _InputIterator);
  for ( ; __first != __last; ++__first)
    __f(*__first);
  return __f;
}
```

4、lambda表达式（重要）

```
//lambda表达式是一个函数，是一个匿名函数
[](int value){
    cout << value << "";
}(100);
cout << endl;

auto f = [](int value){
    cout << value << " ";
};
f(200);
cout << endl;

typedef void(*pFunc)(int);
using pp = void(int);

pFunc f2 = [](int value)->void{
    cout << value << "";
};
f2(4000);
```

直接给lambda表达式传参

使用lambda表达式给f进行复制，然后使用f的时候等价于在使用lambda表达式

严格的指明f2的类型

对于捕获列表的研究

```
不捕获外部的所有变量，那么就不能再lambda表达式中使用外部变量
[](const string &value){
    /* cout << "num = " << num << endl;//error */
    cout << "value = " << value << endl;
}("wangdao");

cout << endl;
[num](const string &value){
    cout << "num = " << num << endl;
    /* cout << "age = " << age << endl;//error,没有捕获age *
    cout << "value = " << value << endl;
}("wangdao");

cout << endl;
[age, &num](const string &value){
    ++num;
    cout << "num = " << num << endl;
    cout << "age = " << age << endl;
    cout << "value = " << value << endl;
}("wangdao");

cout << endl;
//将num使用引用传递，但是其他外部变量使用值传递
[=, &num](const string &value){
    ++num;
    cout << "num = " << num << endl;
    cout << "age = " << age << endl;
    cout << "name = " << name << endl;
    cout << "value = " << value << endl;
}("wangdao");
```

以值传递的方式捕获外部变量（注意：不能修改num的值）

外部变量age是值传递，但是num是引用传递，也就是age不能修改，但是num可以修改

对于外部变量num采用引用传递，可以在lambda中修改num但是其他外部变量使用的是值传递，不能在lambda中进行修改

```
    cout << endl;
    //将num使用值传递，但是其他外部变量使用引用传递
    [&, num](const string &value){
        age = 20;
        name = "ssss";
        cout << "num = " << num << endl;
        cout << "age = " << age << endl;
        cout << "name = " << name << endl;
        cout << "value = " << value << endl;
    }("wangdao");


```

对于外部变量num采用值传递，但是其他外部变量采用引用传递

注意：全局变量可以不在捕获列表中进行捕获，也是可以直接在lambda表达式的函数体中进行使用。

```

int gNum = 100;

void test()
{
    int num = 10;
    int age = 20;
    string name("hello");

    [](const string &value){
        /* cout << "num = " << num << endl;//error */
        cout << "value = " << value << endl;
        cout << "gNum = " << gNum << endl;
    }("wangdao");

```

全局变量

没有捕获也可以直接使用

5、remove_if的用法

```
first = vec.begin();
last = vec.end();
p = func;

ForwardIt remove_if(ForwardIt first, ForwardIt last, UnaryPredicate p)
{
    first = std::find_if(first, last, p);
    if (first != last)
        for(ForwardIt i = first; ++i != last; )
            if (!p(*i))
                *first++ = std::move(*i);
    return first;
}

constexpr InputIt find_if(InputIt first, InputIt last, UnaryPredicate p)
{
    for (; first != last; ++first) {
        if (p(*first)) {
            return first;
        }
    }
    return last;
}
```

```
begin          f        end/last
1, 3, 5, 4, 2, 4, 2, 8
                        i
```

6、vector的扩容导致迭代器失效（重要，坑）

```
void test()
{
    vector<int> vec;
    vec.push_back(10);

    bool flag = true;
    for(auto it = vec.begin(); it != vec.end(); ++it)
    {
        cout << *it << "  ";
        if(flag)
        {
            vec.push_back(20);//底层已经发生了扩容
            flag = false;
            it = vec.begin();//重新置位
        }
    }
    cout << endl;
}
```

Day22

一、问题回顾

1、三组插入迭代器是什么？底层分别调用了什么？

2、什么是一元函数？什么是一元谓词？什么是二元函数？

3、lambda表达式由那几部分构成？捕获列表中捕获外部变量有哪些方式？

```
[捕获列表](函数参数列表)opt ->returnType
{
  //函数体
};

opt:mutable
void test()
{
    int num = 10;
    int age = 20;
    string name("wangdao");

    [&,num](){
        age = 300;
        cout << num << endl;
        cout << age << endl;
        cout << name << endl;
    };
```

```
}
```

二、bind1s与bind2nd

std::bind1st, std::bind2nd

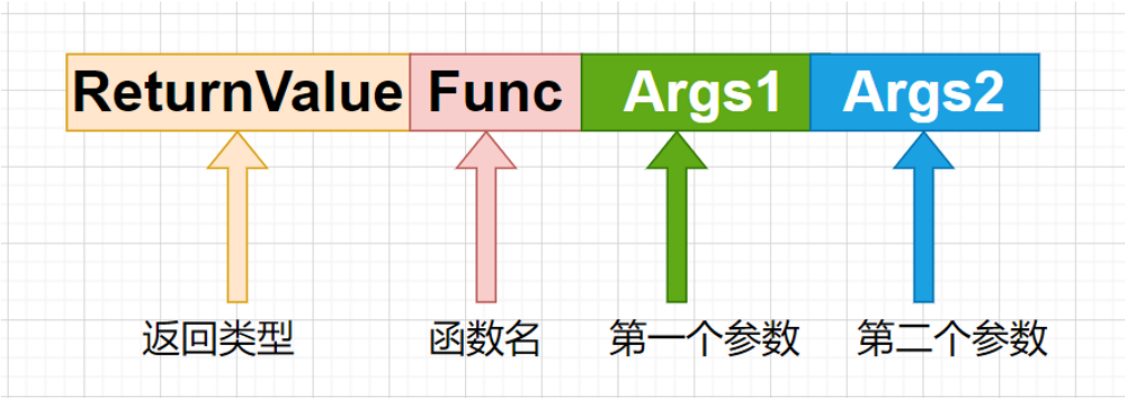
bind1st可以绑定二元函数对象f的第一个参数，bind2nd可以绑定二元函数对象f的第二个参数

Defined in header <functional>

```
template< class F, class T >
std::binder1st<F> bind1st( const F& f, const T& x );
```
(1) (deprecated in C++11) (removed in C++17)

```
template< class F, class T >
std::binder2nd<F> bind2nd( const F& f, const T& x );
```
(2) (deprecated in C++11) (removed in C++17)

Binds a given argument x to a first or second parameter of the given binary function object f. That is, stores x within the resulting wrapper, which, if called, passes x as the first or the second parameter of f.



对于二元函数对象Func而言，如果固定第一个参数Args1为5，那么应该是5 < Args2，也就是二元函数对象的左操作数小于右操作，也就是表明Func具备小于符号的含义；也就是std::less
如果固定的是二元函数对象的第二个参数Args2为5，那么应该是Args1 > 5，也就是二元函数对象应该具备大于符号的含义，也就是std::greater

```
    cout << endl << "执行remove_if操作" << endl;
    auto it = remove_if(vec.begin(), vec.end(),
                        std::bind1st(std::less<int>(), 5));
    /* auto it = remove_if(vec.begin(), vec.end(), */
    /*                     std::bind2nd(std::greater<int>(), 5)); */
    /* auto it = remove_if(vec.begin(), vec.end(), [](int value)->bool{ */
    /*                     return value > 5; */
    /*                     }); */

    //remove_if并不能将满足条件的元素一次性都删除，但是会返回
    //待删除元素的首迭代器，然后再配合容器的erase操作，就可以
    //删除其他多余的元素
    /* auto it = remove_if(vec.begin(), vec.end(), func); */
    vec.erase(it, vec.end());
    for_each(vec.begin(), vec.end(), [](int &value){
            cout << value << "  ";
            });
    cout << endl;
```

三、bind的使用(重要)

1、模板参数

Defined in header <functional>

```
template< class F, class... Args >
/*unspecified*/ bind( F&& f, Args&&... args );
```
(1) (since C++11)

```
template< class R, class F, class... Args >
/*unspecified*/ bind( F&& f, Args&&... args );
```
(2) (since C++11)

The function template **bind** generates a forwarding call wrapper for **f**. Calling this wrapper is equivalent to invoking **f** with some of its arguments bound to **args**.

2、基本用法

```
void test()
{
    //bind可以绑定普通函数，也可以绑定成员函数，函数可以是一元的，
    //也可以是二元的，也可以是n元的
    auto f = bind(add, 1, 3);
    cout << "f() = " << f() << endl;

    cout << endl;
    auto f2 = bind(&multiply, 3, 4, 5);
    cout << "f2() = " << f2() << endl;

    cout << endl;
    Example ex;
    auto f3 = bind(&Example::add, &ex, 10, 20);
    cout << "f3() = " << f3() << endl;
}
```

成员函数一定要传地址，不能传函数名字

```
class Example
{
public:
    //Example * this
    int add(int x, int y)
    {
        cout << "int Example::add(int, int)" << endl;
        return x + y;
    }

    int data = 10;//c++11
};
```

数据成员

bind还可以绑定数据成员

```
    cout << endl;
    function<int()> f5 = bind(&Example::data, &ex);
    cout << "f5() = " << f5() << endl;
```

3、函数指针

```
void test()
{
    /* typedef int(*)() pFunc;//error */
    typedef int(*pFunc)() ;//ok
    using pf = int(*)();//ok

    //先注册回调函数，然后再执行回调函数
    //延迟调用的思想
    pFunc f = &func;//注册
    //....
    //...
    cout << "f() = " << f() << endl;//执行

    f = func2;
    cout << "f() = " << f() << endl;//执行

    /* f = func3;//error */
}
```

4、占位符

```
void func(int x1, int x2, int x3, const int &x4, int x5)
{
    cout << "x1 = " << x1 << endl
         << "x2 = " << x2 << endl
         << "x3 = " << x3 << endl
         << "x4 = " << x4 << endl
         << "x5 = " << x5 << endl;
}

void test()
{
    int number = 100;
    using namespace std::placeholders;
    //占位符本身代表的是形参的位置
    //占位符中的数字代表的事实参的位置
    //bind默认采用的是值传递
    //cref = const reference,引用的包装器
    //ref = reference,引用的包装器
    auto f = bind(func, 10, _4, _3, std::cref(number), number);
    number = 200;
    f(1, 20, 300, 400, 500, 600);//多余的参数直接丢掉
}
```

四、function的使用（重要）

1、模板参数

std::function

Defined in header <functional>

```
template< class >
class function; /* undefined */
```
(since C++11)

```
template< class R, class... Args >
class function<R(Args...)>;
```
(since C++11)

2、使用

```
void test()
{
    //add函数类型(形态、标签)：函数的返回类型 + 参数列表
    //int(int, int)
    //通过bind进行绑定之后，将int(int, int)类型中的两个参数固定
    //了，那么函数的类型就应该是int()
    //function可以用来存函数类型，所以function就是函数的容器
    function<int()> f = bind(add, 1, 3);
    cout << "f() = " << f() << endl;

    cout << endl;
    //int(int, int, int)--> int()
    function<int()> f2 = bind(&multiply, 3, 4, 5);
    cout << "f2() = " << f2() << endl;
```

Day23

一、问题回顾

1、函数指针是什么？什么是回调函数？回调函数的使用包括哪两步？

```
int (*pf)(int, int);
pf = func;//注册
//...
//...
pf(1, 2);//执行
```

2、bind的特点是什么？什么是占位符？占位符有什么特点？默认是值传递还是引用传递？如何进行引用传递？

```
bind(F, ...);
auto f = bind(add, 1, _2);
f(10, 20);
```

3、什么是函数类型？

```
函数的返回类型 + 函数的参数列表
int(int, int);
int(int);
```

4、function的特点是什么?

```
int add(int, int);
function<int(int)> f = bind(add, 1, _1);
```

二、bind的使用（重要）

```
void test()
{
    Example ex;
    function<int()> f = bind(&Example::add, &ex, 10, 20);
    cout << "f() = " << f() << endl;

    cout << endl;
    function<int()> f2 = bind(&Example::add, ex, 100, 200);
    cout << "f2() = " << f2() << endl;
}
```

传地址

传对象，将对象拷贝一份

总结：使用传地址的方式，可以减少内存开销，但是如果ex已经提前销毁了，那么取地址就是一个空指针；使用传对象，也就是会增加内存开销，但是如果老对象已经销毁了，也不会有影响。

三、bind与function的结合使用（重要）

```
//面向对象的思想：继承 + 虚函数
//基于对象的思想：std::bind + std::function

class Figure
{
public:
    using DisplayCallback = function<void()>;
    using AreaCallback = function<double()>;

    //右值引用的特点：只能识别右值，不能识别左值
    void setDisplayCallback(DisplayCallback &&cb)
    {
        _displayCallback = std::move(cb);
    }

    void setAreaCallback(AreaCallback &&cb)
    {
        _areaCallback = std::move(cb);
    }
```

给function取别名

注册

```
30      void handleDisplayCallback() const
31      {
32          if(_displayCallback)
33          {
34              _displayCallback();
35          }
36      }
37
38      double handleAreaCallback() const
39      {
40          if(_areaCallback)
41          {
42              return _areaCallback();
43          }
44          else
45          {
46              return 0.0;
47          }
48      }
49
50      DisplayCallback _displayCallback;//对象
51      AreaCallback  areaCallback;
```

执行

两个对象（数据成员）

```
157     cout << endl;
158     Figure fig;
159     fig.setDisplayCallback(bind(&Rectangle::display, &rectangle, 10));
160     fig.setAreaCallback(bind(&Rectangle::area, &rectangle));
161     func(fig);
162
163     fig.setDisplayCallback(bind(&Circle::print, &circle));
164     fig.setAreaCallback(bind(&Circle::printArea, &circle));
165     func(fig);
166
167     fig.setDisplayCallback(bind(&Triangle::show, &triangle));
168     fig.setAreaCallback(bind(&Triangle::showArea, &triangle));
169     func(fig);
```

bind改变成员函数的形态

```
void func(const Figure &fig)
{
    fig.handleDisplayCallback();
    cout << "的面积 : " << fig.handleAreaCallback() << endl;
}
```

执行注册的回调函数

四、成员函数适配器mem_fn

```
void test()
{
    vector<Number> vec;
    for(size_t idx = 1; idx != 30; ++idx)
    {
        vec.push_back(Number(idx));
    }
                                成员函数适配器

    /* for_each(vec.begin(), vec.end(), mem_fn(&Number::print)); */
    Number num(1);
    /* for_each(vec.begin(), vec.end(), bind(&Number::print, num)); *
    using namespace std::placeholders;
    for_each(vec.begin(), vec.end(), bind(&Number::print, _1));
    cout << endl;
                                利用bind可以改变函数形态的特点
}
```

五、函数对象（仿函数）

任何可以与小括号进行结合体现函数含义的，都可以称为函数对象

- 函数名
- 函数指针
- 重载了函数调用运算符的类创建的对象

六、空间配置器（重要，难）

1、头文件

```
#include <memory>
```

2、特征

会将空间的申请与对象的构建分开，将空间的释放与对象的销毁也分开。在STL中，对象的创建是批量的，有可能一次创建多个对象，如果还使用创建一个对象就申请一块空间，那么空间的申请就会变得非常的频繁，同样的，对象的销毁的时候，也会频繁的回收空间，这样效率比较低，甚至对空间的利用率不高。

3、常用函数

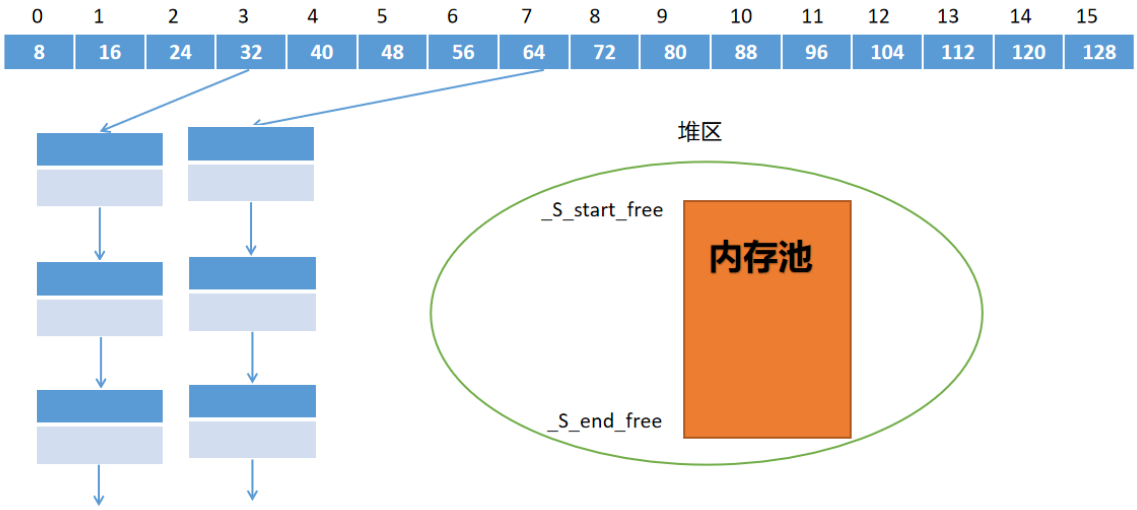
```
//空间申请
T* allocate( std::size_t n );

//空间的释放
void deallocate( T* p, std::size_t n );

//对象构建
void construct( pointer p, const_reference val );

//对象销毁
void destroy( pointer p );
```

4、使用

自定义Vector的实现

5、原理图

第二级空间配置器的原理图



Day24

一、问题回顾

1、bind与function的结合使用?

2、什么是函数对象?

3、空间配置器有几级空间配置器?